

Fedimint Confidential eCash

Gazelle

July 2026

revised September 2026

1 Introduction

Electronic cash (eCash) aims to provide a digital analogue of physical cash, enabling users to perform electronic payments while preserving strong privacy guarantees. Traditional eCash systems focus on anonymity by preventing the issuer from linking the withdrawal of a coin to its subsequent spending through the use of blind signatures. More recent constructions extend this notion of privacy by providing *confidential* eCash, where not only the identity of the user remains protected, but also the value of the eCash note. In such systems, the denomination is cryptographically hidden from all parties except those authorized to learn it, while still allowing the correctness of transactions to be publicly verified.

Designing a confidential blind eCash system requires satisfying several fundamental security and practical properties. First, the system should guarantee **unlinkability**, ensuring that an issued note cannot be linked to the transaction in which it is later spent, thereby preserving the user’s anonymity. Second, it must satisfy **unforgeability**, meaning that every valid eCash note must originate from the mint and that no adversary can create additional spendable notes or increase their value. Third, the protocol should support **offline transactions**, allowing payments to be completed without requiring communication with the mint during the spending phase. Finally, the construction should be **practical**, with cryptographic proofs and transaction sizes that remain compact and computational costs that are sufficiently low for deployment in real-world payment systems.

This document introduces the concept of confidential blind eCash generated within a federation and discusses the cryptographic techniques that enable both transaction privacy and value confidentiality while maintaining strong security guarantees and practical efficiency. Section 3 fixes the notation, Section 4 explains the scheme for a single note, and only afterwards Section 5 introduces the batching of issuance requests as an optimisation on top of it.

2 Threshold Anonymous Credentials

This project builds upon a modified version of the Coconut credential scheme [1], as adapted and deployed by Nym Technologies [2, 3]. Coconut is a threshold anonymous credential system that enables users to obtain credentials from multiple authorities while preserving privacy through selective disclosure and unlinkability. The underlying signature is the Pointcheval–Sanders (PS) signature [4], whose signing operation is linear in the secret key and can therefore be distributed with Shamir secret sharing, and whose signatures can be re-randomised so that a credential can be shown without being recognised.

Nym’s variant differs from the original Coconut construction in the issuance protocol: instead of ElGamal-encrypting the hidden attributes for the authorities, the user sends Pedersen commitments to them in a base derived from a hash. This removes the user’s ElGamal key pair, shrinks the request, and makes unblinding a matter of subtracting public-key material. Nym

also publishes the verification key in both source groups of the pairing so that unblinding can be carried out in $\mathbb{G}_1$. The following sections describe the credential structure adopted in this project, which follows Nym’s variant, and highlight where it deviates from it.

At a high level, a credential consists of a set of cryptographic commitments to user attributes together with a threshold signature issued jointly by a committee of authorities. The commitments allow users to prove statements about their attributes in zero knowledge without revealing the attributes themselves, while the threshold signature guarantees that the credential has been legitimately issued even if only a quorum of authorities participates.

3 Notation and Setting

3.1 Scalars, groups and the pairing

Throughout, $\mathbb{K} := \mathbb{Z}/p\mathbb{Z}$ is the field of integers modulo the prime p , the order of the BLS12-381 groups. Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be the BLS12-381 pairing groups, all of order p and *written additively*, and let

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$$

be the pairing. It is bilinear: for all $a, b \in \mathbb{K}$, $U \in \mathbb{G}_1$ and $W \in \mathbb{G}_2$,

$$e(aU, bW) = ab e(U, W).$$

Since each group has prime order p , scalar multiplication $\mathbb{K} \times \mathbb{G}_i \rightarrow \mathbb{G}_i$, $(\lambda, U) \mapsto \lambda U$ makes every $\mathbb{G}_i$ a one-dimensional $\mathbb{K}$ -vector space. All maps in this document that send attributes and randomness to group elements are therefore $\mathbb{K}$ -linear maps, and the whole scheme can be read in the language of linear algebra. There is no efficient map from $\mathbb{G}_1$ to $\mathbb{G}_2$ or back (type-3 pairing), which is what makes it safe to publish key material in both groups.

Colour legend: every symbol is coloured by the set it lives in

$m, r, x, \rho, s, \gamma, \lambda$	scalars in $\mathbb{K}$ (32 B)	lowercase
$G_1, G_p, H_p, H_j, B, C_j, P, Y_j, \sigma, R$	elements of $\mathbb{G}_1$ (48 B)	uppercase or Greek
G_2, X_j, K', V	elements of $\mathbb{G}_2$ (96 B)	uppercase
$e(\cdot, \cdot)$	elements of $\mathbb{G}_T$	only inside pairing checks

Groups are written additively. Group elements are uppercase letters (or Greek, as for the signature σ); scalars are lowercase. Sizes are for compressed encodings. A scalar times a $\mathbb{G}_1$ element is a $\mathbb{G}_1$ element; a scalar times a $\mathbb{G}_2$ element is a $\mathbb{G}_2$ element; elements of different groups are never added to each other. Vectors are written with an arrow ($\vec{w}$, $\vec{C}$) and are typed component-wise.

3.2 Generators and hash functions

$G_1 \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$ are fixed generators. All further generators are derived by hashing domain-separation strings to the curve (nothing-up-my-sleeve), so that nobody knows a discrete logarithm relation between any two of them:

- $G_p, H_p \in \mathbb{G}_1$ – Pedersen basis for *amounts*;
- $H_1, H_2, H_3 \in \mathbb{G}_1$ – basis for the attribute commitment C_m .

The generator G_1 doubles as the blinding generator of all commitments that the mint signs. This is not a coincidence: unblinding in Section 4.6 only works because the blinding generator of the commitments is the same generator in which the mint’s public key is published in $\mathbb{G}_1$.

Two hash functions, modelled as random oracles, are used:

$$H_{G_1} : \{0, 1\}^* \rightarrow \mathbb{G}_1 \quad \text{and} \quad H_s : \{0, 1\}^* \rightarrow \mathbb{K}.$$

H_{G_1} derives the per-note signature base B ; H_s derives Fiat–Shamir challenges and encodes the spend condition as a scalar. $\parallel$ denotes concatenation of canonical encodings.

3.3 Attributes

Each eCash note carries three attributes in $\mathbb{K}$. They are the messages that the federation signs, and they are never revealed to the mint at issuance.

Attribute	Meaning	Origin
m_1	the <i>amount</i>	a 64-bit integer lifted to $\mathbb{K}$
m_2	the <i>serial secret</i>	uniformly random; basis of the nullifier
$m_3 = H_s(\text{auth})$	the <i>spend condition</i>	hash of authentication data fixed at issuance

3.4 Keys

For a single issuer, the secret key is a vector of four scalars,

$$\text{sk} = (x_0, x_1, x_2, x_3) \in \mathbb{K}^4,$$

one *constant term* x_0 plus one scalar x_j per attribute m_j . The public key publishes each component in *both* source groups:

$$\text{pk} = \left(\underbrace{\{Y_j = x_j G_1\}_{j=0}^3}_{\text{in } \mathbb{G}_1: \text{ used for unblinding}}, \underbrace{\{X_j = x_j G_2\}_{j=0}^3}_{\text{in } \mathbb{G}_2: \text{ used for verification}} \right).$$

In Coconut's notation X_0 is α , X_j is β_j and Y_j is Nym's $\beta_{g_1,j}$ [3]. Section 4.7 replaces the single issuer by a t -of- n federation whose *aggregate* key has exactly this shape.

3.5 Summary of symbols

Table 1 lists every symbol used in the remainder of the document together with the set it lives in.

4 The Scheme for a Single Note

This section presents the lifecycle of *one* eCash note, first with a single issuer and then with a federation of mints. Batching several notes into one request is deferred to Section 5; nothing in this section depends on it.

4.1 The underlying signature

A Pointcheval–Sanders signature [4] on the attributes (m_1, m_2, m_3) with base point $B \in \mathbb{G}_1$ (Coconut's h) is the pair (B, σ) with

$$\sigma = \left(x_0 + \sum_{j=1}^3 x_j m_j \right) B, \tag{1}$$

verified with the public key in $\mathbb{G}_2$ by the single pairing equation

$$e\left(B, X_0 + \sum_{j=1}^3 m_j X_j\right) \stackrel{?}{=} e(\sigma, G_2). \tag{2}$$

Correctness is bilinearity: both sides equal $e(B, G_2)$ raised to $x_0 + \sum_j x_j m_j$. Three properties of this signature carry the whole scheme.

Table 1: Symbols, their types and their meaning.

Symbol	Type	Meaning
$p, \mathbb{K}$	–	group order and scalar field $\mathbb{Z}/p\mathbb{Z}$
G_1, G_2	$\mathbb{G}_1, \mathbb{G}_2$	fixed generators (Coconut's g_1, g_2); G_1 is also the blinding generator
G_p, H_p	$\mathbb{G}_1$	Pedersen basis for amounts
H_1, H_2, H_3	$\mathbb{G}_1$	basis for the attribute commitment
m_1, m_2, m_3	$\mathbb{K}$	amount, serial secret, spend condition
auth	bytes	authentication data with $m_3 = H_s(\text{auth})$
r_p, r_m, r_1, r_2, r_3	$\mathbb{K}$	commitment randomness chosen by the user at issuance
P	$\mathbb{G}_1$	Pedersen commitment to the amount
C_m	$\mathbb{G}_1$	commitment to all attributes; input to $H_{\mathbb{G}_1}$
B	$\mathbb{G}_1$	signature base, $B = H_{\mathbb{G}_1}(C_m)$ (Coconut's h)
C_1, C_2, C_3	$\mathbb{G}_1$	blinded attributes (commitments in base B)
$\vec{C}$	$\mathbb{G}_1^5$	the statement of the issuance proof, (P, C_m, C_1, C_2, C_3)
$\vec{w}$	$\mathbb{K}^8$	the witness of the issuance proof, $(m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3)$
φ_B	$\mathbb{K}^8 \rightarrow \mathbb{G}_1^5$	linear map with $\varphi_B(\vec{w}) = \vec{C}$
$\vec{\rho}, \vec{R}, \gamma, \vec{s}$	$\mathbb{K}^8, \mathbb{G}_1^5, \mathbb{K}, \mathbb{K}^8$	Σ -protocol nonce, commitment, challenge, response
$x_0, \dots, x_3$	$\mathbb{K}$	secret key
$Y_j = x_j G_1, X_j = x_j G_2$	$\mathbb{G}_1, \mathbb{G}_2$	public key in both groups
$\tilde{\sigma}, \sigma$	$\mathbb{G}_1$	blinded and unblinded signature; (B, σ) is the credential
λ_i, S	$\mathbb{K}, \text{set}$	Lagrange coefficients and the set of t contributing mints
r, r', r'_p	$\mathbb{K}$	re-randomisation and fresh commitment randomness at spend time
(B', σ', V)	$\mathbb{G}_1, \mathbb{G}_1, \mathbb{G}_2$	re-randomised signature shown at spend time
P'	$\mathbb{G}_1$	fresh Pedersen commitment to the amount at spend time
K'	$\mathbb{G}_2$	nullifier, $K' = m_1 X_1 + m_2 X_2$
ψ	$\mathbb{K}^3 \rightarrow \mathbb{G}_1 \times \mathbb{G}_2$	linear map of the spend proof

Why PS signatures

1. **Signing is linear in the key.** Equation (1) is a linear function of $(x_0, \dots, x_3)$. Anything linear in the key can be Shamir-shared and interpolated in the exponent, which gives threshold issuance (Section 4.7).
2. **Signing is linear in the messages.** A commitment $m_j B + r_j G_1$ can be fed into (1) in place of $m_j B$; the signer never sees m_j , and the extra term is removable. This gives blind issuance (Section 4.5).
3. **Re-randomisable.** For any $r \neq 0$, $(rB, r\sigma)$ is an equally valid signature on the same messages, and is unlinkable to (B, σ) under DDH in $\mathbb{G}_1$. This gives unlinkable spending (Section 4.8).

The base B must be different for every signature. If two signatures share a base, their affine combinations are again valid signatures, on the corresponding affine combination of the messages (Coconut, footnote 3 in [1]; see also Section 5.3). In plain PS the signer picks B at random, which a set of independent mints cannot do consistently; Coconut instead derives B by hashing a commitment supplied by the user, so that every mint computes the same B on its own.

4.2 Requesting issuance

The user holds the attributes (m_1, m_2, m_3) , samples the randomness r_p, r_m, r_1, r_2, r_3 uniformly from $\mathbb{K}$, and forms five elements of $\mathbb{G}_1$:

$$P = m_1 G_p + r_p H_p \quad (\text{Pedersen commitment to the amount}) \quad (3)$$

$$C_m = m_1 H_1 + m_2 H_2 + m_3 H_3 + r_m G_1 \quad (\text{commitment to all attributes}) \quad (4)$$

$$B = H_{\mathbb{G}_1}(C_m) \quad (\text{derived signature base}) \quad (5)$$

$$C_j = m_j B + r_j G_1 \quad (\text{blinded attributes, } j \in \{1, 2, 3\}) \quad (6)$$

The issuance request consists of the statement $\vec{C} = (P, C_m, C_1, C_2, C_3) \in \mathbb{G}_1^5$, the proof of Section 4.3, and the range proof of Section 4.4. The mint recomputes B from C_m itself; it is not transmitted.

What each element is for

P – the amount commitment. Pedersen commitment in its own basis (G_p, H_p) : perfectly hiding because of r_p , binding under the discrete-logarithm assumption between G_p and H_p . It is *additively homomorphic*, $P(m, r) + P(m', r') = P(m + m', r + r')$, which lets the transaction layer check that inputs and outputs balance without seeing the amounts. It is also the object the range proof (Section 4.4) is attached to. It is deliberately in a basis unrelated to the signature so that balance checking is independent of the credential.

C_m – the commitment behind B . A Pedersen vector commitment to all three attributes with randomness r_m . Its only purpose is to be hashed into B . Because it is *binding*, B is a function of the attributes the user is about to get signed; because it is *hiding*, the mint learns nothing from it; because r_m is fresh, every request gets a fresh base even for identical attributes.

B – the signature base. Neither party chooses it: the user cannot pick a base with a known relation to another note's base (which would enable the affine-combination forgery of Section 4.1), and the mint does not have to pick anything, so all mints of a federation agree on B without communicating. This is the Coconut trick that turns PS into a threshold-friendly scheme.

C_j – the blinded attributes. Pedersen commitments to each attribute *in the base B*

with blinding on G_1 . The base B is what the signature (1) multiplies the attributes with, so the mint can apply its key directly to C_j (Section 4.5). The blinding generator G_1 is what the public key Y_j is published in, so the blinding term the mint inadvertently signs, $r_j x_j G_1 = r_j Y_j$, is computable by the user from public data. In the original Coconut these were ElGamal ciphertexts of $m_j B$; Nym's commitments serve the same purpose with less data.

4.3 The issuance proof

The mint must be convinced that $\vec{C}$ is well formed before signing: otherwise a user could, for instance, put one amount into P (and its range proof) and a different amount into C_1 (and the signature). Well-formedness of $\vec{C}$ is exactly the statement that $\vec{C}$ lies in the image of the linear map $\varphi_B : \mathbb{K}^8 \rightarrow \mathbb{G}_1^5$,

$$\varphi_B(m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3) = \begin{pmatrix} m_1 G_p + r_p H_p \\ m_1 H_1 + m_2 H_2 + m_3 H_3 + r_m G_1 \\ m_1 B + r_1 G_1 \\ m_2 B + r_2 G_1 \\ m_3 B + r_3 G_1 \end{pmatrix} = \begin{pmatrix} P \\ C_m \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \vec{C}. \quad (7)$$

The witness is

$$\vec{w} = (m_1, m_2, m_3, r_p, r_m, r_1, r_2, r_3) \in \mathbb{K}^8.$$

Note that φ_B depends on B , which the verifier recomputes from the second component of $\vec{C}$; the map is fixed once C_m is.

The user proves knowledge of a preimage with the standard Σ -protocol for linear maps (Schnorr's protocol generalised to a vector-valued linear map), made non-interactive with the Fiat-Shamir transform under the domain tag $\text{tag}_{\text{iss}} = \text{FEDIMINT_ECASH_CHALLENGE_ISSUANCE}$:

$$\begin{aligned} \vec{\rho} &\xleftarrow{\$} \mathbb{K}^8, & \vec{R} &= \varphi_B(\vec{\rho}) \in \mathbb{G}_1^5, \\ \gamma &= H_s(\text{tag}_{\text{iss}} \parallel \vec{C} \parallel \vec{R}) \in \mathbb{K}, & \vec{s} &= \vec{\rho} + \gamma \vec{w} \in \mathbb{K}^8. \end{aligned} \quad (8)$$

The proof is $(\vec{R}, \vec{s})$: five $\mathbb{G}_1$ elements and eight scalars, $5 \cdot 48 + 8 \cdot 32 = 496$ bytes. The verifier recomputes B and γ and checks

$$\varphi_B(\vec{s}) \stackrel{?}{=} \gamma \vec{C} + \vec{R}. \quad (9)$$

Why this proves what we need

Completeness. By linearity, $\varphi_B(\vec{s}) = \varphi_B(\vec{\rho}) + \gamma \varphi_B(\vec{w}) = \vec{R} + \gamma \vec{C}$.

Knowledge (special soundness). Two accepting transcripts with the same $\vec{R}$ and different challenges $\gamma \neq \gamma'$ give $\varphi_B(\vec{s} - \vec{s}') = (\gamma - \gamma') \vec{C}$, so $(\gamma - \gamma')^{-1}(\vec{s} - \vec{s}')$ is a witness. A prover who can answer two challenges knows a preimage. In the random-oracle model Fiat-Shamir preserves this.

Zero knowledge. A simulator picks $\vec{s} \xleftarrow{\$} \mathbb{K}^8$ and $\gamma \xleftarrow{\$} \mathbb{K}$ first and sets $\vec{R} = \varphi_B(\vec{s}) - \gamma \vec{C}$; the transcript is distributed exactly like a real one. The mint learns nothing about the attributes beyond the commitments, which are hiding.

What the single witness buys us. One witness vector serves all five equations, so the same m_1 sits in P , C_m and C_1 , and the same m_2, m_3 sit in C_m and C_2, C_3 . Hence (i) the amount that is range-proven is the amount that gets signed (unforgeability of value), (ii) B really is derived from a commitment to the attributes being signed (the freshness argument for B holds), and (iii) the C_j are of the exact form (6), so the blinding term

in the signature is exactly $\sum_j r_j Y_j$ and nothing else gets signed. Hashing $\vec{C}$ into the challenge binds the proof to the statement; the domain tag prevents replaying a proof for ψ of Section 4.8 against φ_B .

4.4 Range proof on the amount

The commitment P hides m_1 as an element of $\mathbb{K}$, and $\mathbb{K}$ wraps around: a “negative” amount $-v$ is the field element $p - v$. Without further constraints an adversary could commit to an out-of-range value and satisfy a balance equation $\sum P_{\text{in}} = \sum P_{\text{out}}$ while creating value from nothing. The user therefore additionally proves that the committed amount lies in an accepted interval, e.g. $0 \leq m_1 < 2^{64}$. This work employs Bulletproofs [5], which give short non-interactive range proofs over Pedersen commitments without a trusted setup, with proof size logarithmic in the bit length and aggregatable across several commitments. Because the issuance proof ties the m_1 in P to the m_1 in the signature, the range proof on P carries over to the signed amount.

4.5 Blind signing

A mint that has verified the proofs recomputes $B = H_{\mathbb{G}_1}(C_m)$ and applies its key to the request as if the C_j were the attributes:

$$\tilde{\sigma} = x_0 B + \sum_{j=1}^3 x_j C_j = \underbrace{\left(x_0 + \sum_{j=1}^3 x_j m_j \right) B}_{\text{PS signature } \sigma, \text{ eq. (1)}} + \underbrace{\sum_{j=1}^3 r_j (x_j G_1)}_{\text{blinding term} = \sum_j r_j Y_j}. \quad (10)$$

The expansion is just (6) substituted into the first expression. The mint never sees m_j ; it sees B , the C_j and the proof, all of which are independent of the attributes by the hiding property and zero knowledge.

4.6 Unblinding and verification

The blinding term in (10) is a linear combination of the $\mathbb{G}_1$ public key components with coefficients the user chose, so the user removes it from public information only:

$$\sigma = \tilde{\sigma} - \sum_{j=1}^3 r_j Y_j, \quad (11)$$

and then checks (B, σ) against the $\mathbb{G}_2$ key with the PS equation (2). This is where the choice of G_1 as the blinding generator in (6) is essential: with any other blinding generator the user would need x_j to cancel the term. The pair (B, σ) together with (m_1, m_2, auth) is the eCash note.

4.7 Threshold issuance

Nothing above requires a single issuer, because (10) is $\mathbb{K}$ -linear in the secret key $(x_0, \dots, x_3)$. The standard threshold-BLS transformation therefore applies component-wise.

Key sharing. Each x_j is Shamir-shared: pick random polynomials $f_j \in \mathbb{K}[X]$ of degree at most $t - 1$ with $f_j(0) = x_j$, and give mint $i \in \{1, \dots, n\}$ the key share

$$\text{sk}_i = (f_0(i), f_1(i), f_2(i), f_3(i)) \in \mathbb{K}^4,$$

with public key shares $Y_{i,j} = f_j(i) G_1$ and $X_{i,j} = f_j(i) G_2$. The aggregate public key is $Y_j = f_j(0) G_1$, $X_j = f_j(0) G_2$, i.e. exactly the single-issuer key of Section 3. Fewer than t shares reveal nothing about x_j .

Share signing and share verification. Every mint runs Section 4.5 with its own share,

$$\tilde{\sigma}_i = f_0(i) B + \sum_{j=1}^3 f_j(i) C_j,$$

and the user unblinds each share with the mint's $\mathbb{G}_1$ key share, $\sigma_i = \tilde{\sigma}_i - \sum_j r_j Y_{i,j}$, then verifies it with (2) against $X_{i,j}$. A mint that returns garbage is identified immediately and simply left out.

Aggregation. Let S be a set of t mint indices whose shares verified. The federation signature is the Lagrange combination at 0,

$$\sigma = \sum_{i \in S} \lambda_i \sigma_i, \quad \lambda_i = \prod_{\substack{k \in S \\ k \neq i}} \frac{k}{k - i} \pmod{p}. \quad (12)$$

Since $\sigma_i = (f_0(i) + \sum_j m_j f_j(i)) B$ and $\lambda \mapsto \lambda B$ is linear, the sum evaluates the polynomial $f_0 + \sum_j m_j f_j$ at 0 inside the group, which is exactly the single-issuer signature (1). Everything downstream is unchanged and verifies against the aggregate key.

4.8 Spending

To spend, the user must show the mint a valid signature on a note without letting the mint connect it to the issuance it came from, while still letting the mint (i) check the amount against the transaction balance and (ii) detect a second spend of the same note.

Re-randomisation. The user samples $r, r' \xleftarrow{\$} \mathbb{K}$ and forms

$$(B', \sigma', V) = (r B, r \sigma + r r' B, r' G_2). \quad (13)$$

The pair $(r B, r \sigma)$ is the PS re-randomisation of Section 4.1: fresh group elements, unlinkable to (B, σ) under DDH in $\mathbb{G}_1$. The r' terms are inherited from Coconut's (κ, ν) and are discussed in Section 4.8.1.

Shown elements. Along with (B', σ', V) and the authentication data `auth` the user sends

$$P' = m_1 G_p + r'_p H_p \in \mathbb{G}_1, \quad K' = m_1 X_1 + m_2 X_2 \in \mathbb{G}_2, \quad (14)$$

where $r'_p \xleftarrow{\$} \mathbb{K}$ is *fresh* commitment randomness, together with a second Fiat–Shamir Σ -proof (tag `tagsp` = `FEDIMINT_ECASH_CHALLENGE_SPEND`) of knowledge of (m_1, m_2, r'_p) for the linear map

$$\psi : \mathbb{K}^3 \rightarrow \mathbb{G}_1 \times \mathbb{G}_2, \quad \psi(m_1, m_2, r'_p) = (m_1 G_p + r'_p H_p, m_1 X_1 + m_2 X_2) = (P', K'). \quad (15)$$

The protocol is the same as in (8) with ψ in place of φ_B and witness (m_1, m_2, r'_p) ; the prototype transmits the challenge instead of the commitment $(R_{P'}, R_K)$, so a spend proof is four scalars (128 bytes), and the verifier recomputes the commitment as $\psi(\vec{s}) - \gamma(P', K')$.

Verification. The mint recomputes $m_3 = H_s(\text{auth})$, requires $m_3 \neq 0$ and $B' \neq 0$, verifies the ψ -proof, checks that K' has not been seen before, and finally checks the signature:

$$e(B', X_0 + K' + m_3 X_3 + V) \stackrel{?}{=} e(\sigma', G_2). \quad (16)$$

Expanding both sides shows why it holds. Write $z := x_0 + x_1 m_1 + x_2 m_2 + x_3 m_3$, so that $\sigma = z B$ and $X_0 + K' + m_3 X_3 = z G_2$. Then

$$\begin{aligned} \text{LHS} &= e(rB, (z + r') G_2) = r(z + r') e(B, G_2), \\ \text{RHS} &= e(rz B + rr' B, G_2) = r(z + r') e(B, G_2). \end{aligned}$$

The equation is the PS verification (2) with the hidden part $m_1 X_1 + m_2 X_2$ of the message supplied as the single element K' , the revealed part $m_3 X_3$ recomputed by the mint, and the r' terms cancelling on both sides.

What each shown element achieves

(B', σ') – **unforgeability, unlinkability.** A valid re-randomised PS signature proves the federation signed *some* attributes; fresh randomness makes the pair independent of the issuance transcript.

K' – **the nullifier and the hidden message.** It supplies the hidden attributes to the verification equation without revealing them, and it is *deterministic* in (m_1, m_2) : spending the same note twice produces the same K' , so the mints keep a set of seen K' and reject repeats. The random m_2 makes K' unique per note and, since m_2 was only ever seen blinded, unlinkable to issuance. Its soundness rests on the ψ -proof: a spender must *know* (m_1, m_2) behind K' , so K' cannot be re-randomised into a fresh nullifier for the same note.

P' – **confidential amount at spend time.** A fresh Pedersen commitment to the same amount, in the balance-check basis, with new randomness so it is unlinkable to P . The ψ -proof ties the m_1 in P' to the m_1 in K' , and (16) ties the m_1 in K' to the signature. So the amount the transaction layer balances with is the amount the federation signed, which is the amount that was range-proven at issuance.

m_3, auth – **the spend condition.** m_3 is baked into σ , so only a spender who can present the committed auth makes (16) hold. Revealing auth does not link the spend to issuance because the mint only saw m_3 blinded. Requiring $m_3 \neq 0$ excludes the degenerate case in which the third attribute contributes nothing to the signature.

4.8.1 On the role of V

In Coconut the spender shows $\kappa = X_0 + \sum_j m_j X_j + r' G_2$ and $\nu = r' B'$, and the verifier checks $e(B', \kappa) = e(\sigma' + \nu, G_2)$. The fresh r' in κ is what hides the attributes there. In this scheme $V = r' G_2$ plays the part of κ 's blinding term and $rr' B$ inside σ' plays the part of ν . However, K' is deliberately *not* blinded, because it has to be a deterministic nullifier; and $e(B', V) = e(rr' B, G_2)$ is computable by anyone from the shown data, so an observer can cancel the r' contribution from $e(\sigma', G_2)$ and recover the same $\mathbb{G}_T$ values as if $r' = 0$. Under this reading V adds 96 bytes and one scalar multiplication but no additional hiding; the unlinkability of a spend rests on the re-randomisation by r and on the fresh r'_p . Whether V should be kept is an open question (Section 8).

4.9 Security properties

Property	Where it comes from
Unforgeability	Existential unforgeability of PS signatures (LRSW-type assumption [4]) with a per-note base $B = H_{\mathbb{G}_1}(C_m)$; binding of P , C_m , C_j ; soundness of the φ_B -proof (same m_1 everywhere) and of the ψ -proof; the range proof on P . Fewer than t mints cannot sign.
Blindness	The mint sees only hiding commitments $\vec{C}$ and zero-knowledge proofs; the blinded signature reveals nothing further.
Unlinkability	Re-randomisation ($rB, r\sigma$) (DDH in $\mathbb{G}_1$) and fresh r'_p in P' make the spend transcript independent of the issuance transcript; m_2 makes K' independent of anything shown at issuance.
Double-spend detection	Equality of nullifiers K' , with the ψ -proof preventing a spender from producing a second nullifier for the same (m_1, m_2) .
Value confidentiality	P and P' are perfectly hiding; K' hides m_1 under DDH in $\mathbb{G}_2$ because of the random m_2 .
Offline spending	Verification (16) needs only the aggregate public key; only double-spend detection needs the mints' shared state.

5 Batching Issuance Requests

A naive approach to requesting several notes at once repeats Section 4.2 per note. The proof alone is then 496 bytes per note, and together with Fedimint's consensus message limit of 64 KB this severely restricts how many notes a single transaction can issue. Several techniques exist to combine multiple Σ -protocol instances into one compact proof: folding as used by Bulletproofs for non-linear relations, or amortisation for parallel instances of the same linear relation. The latter is used here. There are still open discussions concerning the security of the protocol when batching is applied, both for the proofs and for the credential scheme itself; see Section 5.3.

5.1 Batched issuance proof

Suppose the user requests n notes, $i = 1, \dots, n$, with witnesses $\vec{w}_i \in \mathbb{K}^8$ and statements

$$\vec{C}_i = (P_i, C_{m,i}, C_{1,i}, C_{2,i}, C_{3,i}) \in \mathbb{G}_1^5.$$

The amortised protocol sums n instances of (8) into a single response, which only works if all instances use the *same* linear map. Since φ_B depends on B , the batch shares one base derived from all attribute commitments of the batch:

$$B = H_{\mathbb{G}_1}(C_{m,1} \parallel C_{m,2} \parallel \dots \parallel C_{m,n}), \quad \varphi_B(\vec{w}_i) = \vec{C}_i \quad \text{for all } i. \quad (17)$$

For $n = 1$ this coincides with (5). The prover samples one nonce vector per instance, commits to their sum, derives one challenge *per instance*, and sums the responses:

$$\vec{\rho}_i \xleftarrow{\$} \mathbb{K}^8, \quad \vec{R}_i = \varphi_B(\vec{\rho}_i), \quad \vec{R} = \sum_{i=1}^n \vec{R}_i \in \mathbb{G}_1^5, \quad (18)$$

$$\gamma_i = H_s(\text{tag}_{\text{iss}} \parallel \vec{C}_1 \parallel \dots \parallel \vec{C}_n \parallel \vec{R} \parallel i) \in \mathbb{K}, \quad (19)$$

$$\vec{s} = \sum_{i=1}^n (\vec{\rho}_i + \gamma_i \vec{w}_i) \in \mathbb{K}^8. \quad (20)$$

The prover sends $(\vec{R}, \vec{s})$, again 496 bytes regardless of n , along with the statements $\vec{C}_1, \dots, \vec{C}_n$. The verifier recomputes B from (17) and the γ_i from (19) and checks

$$\varphi_B(\vec{s}) \stackrel{?}{=} \sum_{i=1}^n \gamma_i \vec{C}_i + \vec{R}. \quad (21)$$

Why the batched proof works

Completeness. Linearity of φ_B applied to (20): $\varphi_B(\vec{s}) = \sum_i \varphi_B(\vec{\rho}_i) + \sum_i \gamma_i \varphi_B(\vec{w}_i) = \vec{R} + \sum_i \gamma_i \vec{C}_i$. This step is exactly where all instances must share φ_B , i.e. share B .

Soundness. Equation (21) is the single-instance check (9) for the *random linear combination* $\sum_i \gamma_i \vec{C}_i$ of the statements. Because every γ_i is derived after all $\vec{C}_i$ and $\vec{R}$ are fixed, and the γ_i are distinct, a prover who does not know a preimage of some $\vec{C}_i$ can only pass with negligible probability; this is the standard small-exponent/random-combination argument for batch verification of linear relations. Including the index i in the hash gives each instance its own challenge, which prevents a witness of one instance from being moved to another.

Zero knowledge. Unchanged: $\vec{s}$ is a sum of uniformly random $\vec{\rho}_i$ plus witness terms, hence uniform.

Range proofs batch independently: Bulletproofs aggregate the n commitments $P_1, \dots, P_n$ into one proof of size logarithmic in n .

5.2 Blind signing a batch

Each mint verifies the batched proof once and then signs every note of the batch with the *shared* base, $\tilde{\sigma}_i = x_0 B + \sum_j x_j C_{j,i}$, exactly as in (10); unblinding, share verification, aggregation and spending are per note and unchanged from Section 4. Note that after Section 4.8 the notes of one batch are shown with independent re-randomisations rB , so the shared base is not visible at spend time.

5.3 The cost of a shared base

Sharing B across a batch is what makes (21) possible, but it removes the guarantee of Section 4.1 that every signature has its own base. All notes of a batch are PS signatures on the same B , and PS signatures on a common base are closed under affine combination: for any scalars λ_i with $\sum_i \lambda_i = 1$,

$$\sum_i \lambda_i \sigma_i = \left(x_0 + \sum_{j=1}^3 x_j \sum_i \lambda_i m_{j,i} \right) B$$

is a valid signature on the attribute vector $\sum_i \lambda_i \vec{m}_i$. This is the forgery Coconut’s footnote 3 warns about [1]. For the present scheme it means that a user holding two notes of one batch with the same spend condition ($m_{3,1} = m_{3,2}$) and different amounts can derive a signature on an arbitrary amount $\lambda m_{1,1} + (1 - \lambda) m_{1,2}$ with a fresh serial secret, and the range proofs of the original notes say nothing about it. Whether this is exploitable end to end depends on the transaction layer (e.g. whether amounts are range-proven again when spent), and the analysis, as well as possible mitigations such as per-note bases with per-instance responses for the B -dependent components of φ_B , is listed in Section 8. Until it is settled, the batched issuance should be regarded as a proof-size optimisation whose security is not yet established.

6 Benchmarks

This section evaluates the performance of the proposed confidential federated eCash protocol through a series of benchmarks covering its main operations. The analysis considers the computational cost incurred by both the prover and the verifier, the size of the generated proofs, and the storage requirements of an eCash note throughout its lifecycle. Additionally, implementation measurements are presented for each protocol phase to assess the protocol’s practicality and identify its computational bottlenecks.

6.1 Proof sizes

The proof sizes differ significantly between the issuance and spending phases due to the different security requirements of each operation. During issuance, the user must prove the correctness of several committed values while preserving their confidentiality. When multiple eCash notes are requested simultaneously, these proofs are batched as in Section 5, which keeps the Σ -proof at a constant 496 bytes independent of the number of notes; only the statements $\vec{C}_i$ grow linearly.

In contrast, the spending phase only requires the user to demonstrate ownership of a valid eCash note. This is the short Σ -proof for ψ , consisting of the challenge and three responses, so the spending proof is compact but still grows linearly with the number of notes being spent. Table 2 shows the proof sizes for each phase, with $\mathbb{G}_1$ and $\mathbb{G}_2$ denoting group elements of the respective groups and s a 32-byte scalar. For an example transaction with 100 eCash notes as both inputs and outputs, the complete transaction (proofs plus the statements and shown elements of every note) takes approximately 72 KB, only slightly over Fedimint’s 64 KB consensus limit.

Table 2: Proof size as a function of the number of eCash notes n . The range proofs are assumed to be for a 32-bit range.

Phase	Proof size	Growth
Issuance (batched Σ -proof)	$5 \mathbb{G}_1 + 8 s$	$O(1)$
Issuance (batched range proofs)	$2 \log_2(32 \times n) \mathbb{G}_1 + 9 \mathbb{G}_1 + 2 s$	$O(\log_2 n)$
Spending	$4 \times n \times s$	$O(n)$

7 Implementation Notes

The prototype in this repository implements Sections 4 and 5 over BLS12-381, with the following deviations from the description that are worth knowing about.

Prototype caveats

- **Spend-time commitment randomness.** The prototype reuses the issuance randomness r_p when building P' , so $P' = P$ and a spend is trivially linkable to its issuance. The scheme requires a fresh r'_p as in (14).
- **Hash-to-group.** $H_{\mathbb{G}_1}$, H_s and the generators are implemented by seeding a ChaCha RNG with SHA-256 rather than a standard `hash_to_curve` ciphersuite. This is fine for unknown-discrete-logarithm generation but not interoperable.
- **Checks not implemented.** $m_3 \neq 0$, $B' \neq 0$ and the nullifier set are not enforced by the prototype; the range proof is not integrated.
- **Only the batched path is exercised.** Single-note issuance is the $n = 1$ case of Section 5, and the tests run batches of two.

8 Future Work

1. Add security assumptions made in the scheme (LRSW/PS unforgeability, DDH in $\mathbb{G}_1$ and $\mathbb{G}_2$, random oracles for $H_{\mathbb{G}_1}$, H_s) and a proof sketch.
2. Add the description and design of the peg-in and peg-out operations when joining the federation.
3. Analyse the affine-combination attack of Section 5.3 enabled by the shared base B in a batched issuance, and its mitigations (per-note bases, range proofs at spend time, or a batching strategy that keeps distinct bases).
4. Add and check the bounds provided for proof batching (soundness error of (21)).
5. Decide whether V and the r' terms of (13) are needed (Section 4.8.1).
6. Fix the prototype deviations of Section 7.
7. Add acknowledgments.

References

- [1] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2019. <https://arxiv.org/abs/1802.07344>.
- [2] Nym Technologies SA, “The Nym network.” <https://nymtech.net/nym-whitepaper.pdf>, 2021. Accessed: 2026-07-28.
- [3] Nym Technologies SA, “nym_offline_compact_ecash,” Rust crate in the `nymtech/nym` repository (`common/nym_offline_compact_ecash`), 2024. Accessed: 2026-09-16.
- [4] D. Pointcheval and O. Sanders, “Short randomizable signatures,” in *CT-RSA 2016*, LNCS 9610, Springer, 2016.
- [5] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, “Bulletproofs: Short proofs for confidential transactions and more,” in *IEEE Symposium on Security and Privacy*, 2018.